Phase 11: Interview Revision Sheet

How to use this: read the pitch out loud until it sounds natural. Then, for each phrase below, cover the answer and explain it in **~30 seconds** in your own words. Then try the follow-ups. Each section points to the phase notes with the full theory.

The resume bullet

> **Real-Time Trading Dashboard & Option Chain** | React, TypeScript, WebSockets - Built a trading web app with a **live watchlist**, **order screen** (Market/Limit with validation), **positions with live P&L** and **candlestick charts**, streaming ticks from the **Binance WebSocket API**. - Engineered a **resilient WebSocket client** with **heartbeat**, **exponential-backoff reconnect** and **symbol resubscription**; batched ticks with **requestAnimationFrame** to hold **60 FPS at 100+ updates/sec**. - Rendered a **200+ strike option chain** (**OI, IV, Black–Scholes Greeks**) using **virtualization (react-window)** and **memoized rows**; kept an **incremental order book** in **sorted maps** with **sequence-gap resync**. - Turned **Figma designs** into a **responsive dark-mode UI**; tested P&L and order logic with **Vitest + React Testing Library** and an order flow with **Playwright** on **GitHub Actions CI**.

---

1. The 60-second pitch (“Tell me about this project”)

> "It’s a real-time crypto trading dashboard in React and TypeScript. It streams live prices from Binance’s public WebSocket API into a watchlist, candlestick charts and a live order book. There’s a simulated order screen with Market and Limit orders, validated against the exchange’s real tick size and lot rules, and positions with live P&L.
>
> The interesting engineering is the real-time side. I wrote a WebSocket client that detects dead connections with a heartbeat, reconnects with exponential backoff and jitter, and resubscribes to its streams. Ticks are batched per animation frame, so the UI renders at most 60 times a second however fast messages arrive. The order book follows Binance’s snapshot-plus-diff algorithm with sequence-gap detection, and the option chain prices 200+ strikes with Black–Scholes on every tick, virtualized with react-window.
>
> It’s responsive with a mobile tab layout and a dark/light theme. It has about 77 tests across Vitest, React Testing Library and Playwright, and a GitHub Actions pipeline that runs them and deploys to GitHub Pages."

Be upfront about these two points (it builds trust): - **Orders are simulated.** Real Binance orders need authenticated API keys and a backend to keep them secret. Validation uses Binance’s real filters, and execution is a local matcher. - **Option IV and OI are modelled.** The spot price is live, Black–

Scholes and the Greeks are real maths, and IV is a modelled smile around a typical volatility. OI is generated deterministically. “A production version would take IV and OI from an options feed like Deribit.”

2. Architecture in one picture

```
Binance WS ──> BinanceSocket (heartbeat · backoff · resubscribe) ──> FrameBatcher
(rAF) ──> Zustand stores ──> React components
Binance REST -> candle history, order-book snapshot
| selectors: each component

| re-renders only for its slice
  marketStore  (tickers, connection status)        tradingStore (cash, positions,
orders; persisted to localStorage)
  uiStore      (theme, mobile tab)                 pure logic: trading/*.ts
(validation, P&L, Black–Scholes) → unit-tested
```

Key design idea: networking and maths live in **plain TypeScript classes and functions** (`BinanceSocket` , `LocalOrderBook` , `portfolio.ts`). React only renders. That’s why most of it is unit-testable without a browser.

---

3. Phrase by phrase

“React, TypeScript” → notes 00, 01

30 s: React builds the UI from **components**: functions that take **props** and **state** and return JSX. When state changes, React re-renders and updates only the DOM that changed. TypeScript adds static types to JavaScript, so wrong shapes and typos are caught at compile time. I typed every Binance message and every store, which made refactors safe. **Follow-ups:** - *What triggers a re-render?* A state change, a parent re-render, or a context/store value it subscribes to changing. - *Why* `key` *in lists?* It gives each item an identity, so React matches old and new items correctly. Don’t use the array index for lists that reorder. - `useEffect` *cleanup?* It runs before the effect re-runs and on unmount. I use it to close sockets and cancel animation frames, which prevents leaks and duplicate connections. - `interface` *vs* `type` *?* They’re mostly interchangeable. `type` can also express unions (`'BUY' | 'SELL'`), which I used for sides, order types and statuses.

“live watchlist” → notes 01, 03

30 s: Eight USDT pairs subscribed over one WebSocket (`<symbol>@ticker` streams). Each row shows price, 24h change and volume, and **flashes green or red** on every price move. Rows are `React.memo` and read their own ticker from the Zustand store with a **selector**, so a BTC tick re-renders only the BTC row. Clicking a row selects the symbol for the chart, order book and option chain. **Follow-ups:** - *Why does only one row re-render?* Zustand compares the selector result. The other rows’ slices didn’t

change, so they're skipped. - *How does the flash work?* The store records each ticker's direction (up/down vs the previous price). The price `<span>` gets `key={price}`: a new key makes React mount a fresh element, which restarts the CSS flash animation. Same price → same key → no flash.

"order screen (Market/Limit with validation)" → notes 05

30 s: Market orders fill now at the current price. **Limit** orders rest until the market reaches your price, then fill at *your* price. The form is **controlled inputs**, validated as you type against Binance's real filters: the **tick size** (price step), the **step size and min quantity**, the **min notional** (≥ 5 USDT), and the available balance. The store **re-validates** on submit, because you never trust the UI layer. **Follow-ups:** - *Why validate twice?* The UI gives fast feedback. The store (or server) is the source of truth, because anything could call it. - *What is "available" cash?* Cash minus the amount **reserved** by open buy limits, so you can't spend the same money twice. - *Floating-point problems?* `0.1 + 0.2 ≠ 0.3`. Check step multiples with a tolerance or integer maths, and round only for display. In a real system, use decimal/integer minor units, like `BigDecimal` in Java. - *Marketable limit?* A buy limit above the market fills right away at the better market price.

"positions with live P&L" → notes 05

30 s: A position stores quantity and **average entry price**. Buying more updates the weighted average. Selling locks in **realized P&L** = (sell price − avg) × qty. **Unrealized P&L** = (current price − avg) × qty and moves with every tick. The maths are pure functions, unit-tested, and the account persists to localStorage. **Follow-ups:** - *Does selling change the average price?* No, it only realizes P&L on the sold part. - *Why pure functions?* Same input, same output, no side effects: trivial to test and reason about.

"candlestick charts" (TradingView Lightweight Charts) → notes 04

30 s: Each candle is **OHLC** (open, high, low, close) for one interval. History comes from **REST** (`/api/v3/klines`, 500 candles) and live updates from the `@kline_<interval>` **WebSocket stream**. Lightweight Charts draws on a **canvas**, and I drive it imperatively through a `useRef`: `series.update()` changes only the last candle instead of re-rendering React. **Follow-ups:** - *Why REST and WebSocket together?* REST for the past (request/response), WebSocket for the present (push). - *Race condition when switching symbol/interval?* The effect's cleanup calls `AbortController.abort()`, so the old request is cancelled and a late answer can't draw over the new chart. - *Why a ref?* The chart is a non-React library that owns its DOM/canvas. React gives us the element, and we call its API directly.

"Binance WebSocket API" → notes 02

30 s: HTTP is request/response: the client always asks. A **WebSocket** starts as an HTTP request that's **upgraded** to a persistent, two-way connection, so the server **pushes** data the moment it changes. Binance's public market streams need no API key. I send `SUBSCRIBE` / `UNSUBSCRIBE` JSON messages on one connection and get JSON events back. **Follow-ups:** *Why not polling?* Latency, wasted requests and rate limits. *Server-Sent Events?* One-way server→client over HTTP. Fine for feeds, but we also send subscribe messages.

“resilient WebSocket client” → notes 02

30 s: A plain TypeScript class, `BinanceSocket`, that hides connection problems from the rest of the app. It has three parts, below: heartbeat, backoff reconnect and resubscription. It also **rotates to a fallback endpoint** (`data-stream.binance.vision`) when the main host is blocked, and emits a status (`connecting / live / reconnecting`) that drives the connection badge. There’s a “simulate drop” button to demo it.

“heartbeat”

30 s: A connection can die **silently**: laptop sleep, Wi-Fi switch, NAT timeout. The browser may say it’s open for minutes while no data arrives. Every **5 s** I check the time since the last message. After **10 s** of silence I send a cheap request as an app-level **ping**, and its reply is the pong. After **20 s** I declare it dead and reconnect. **Follow-up:** *Why not use WebSocket ping frames?* The browser answers protocol pings automatically, but JavaScript can’t send or observe them.

“exponential-backoff reconnect”

30 s: The retry delay doubles each time: 1 s, 2 s, 4 s … capped at **30 s**, and it resets after a successful connection. **Jitter** randomizes each delay (50–100%). Otherwise, when a server restarts, thousands of clients reconnect at the same instant: the **thundering herd**. **Follow-up:** *Why cap the delay?* So recovery stays reasonably quick after a long outage.

“symbol resubscription”

30 s: Subscriptions belong to a connection, so a new socket starts with **none**. The client keeps the **desired set** of streams (a `Set`) as its source of truth. On every (re)connect it sends one `SUBSCRIBE` with all of them. Components add or remove streams any time, even while disconnected, and the set is replayed when the connection comes back.

“batched ticks with requestAnimationFrame to hold 60 FPS at 100+ updates/sec” → notes 03

30 s: A 60 Hz screen paints every **16.6 ms**. Rendering React on every message (100+ per second) is wasted work, because nobody sees the extra frames, and it causes jank. My `FrameBatcher` puts each message in a `Map` keyed by symbol (**coalescing**: only the latest per symbol is kept) and books **one** `requestAnimationFrame`. Right before the next paint, the whole batch goes to the store in **one update → one render per frame at most**. **Follow-ups:** - *Why rAF and not* `setTimeout(16)` *?* rAF is synced to the display’s refresh and paused in background tabs. Timers drift and keep running. - *Where did “100+ updates/sec” come from?* Watchlist tickers plus the order book’s `@depth@100ms` diffs (10/s per symbol) plus klines. The header’s “msg/s → renders/s” counter shows renders staying ≤ 60. - *What if processing itself is slow?* Move parsing to a Web Worker, and virtualize big lists. - *React 18+ already batches setState?* It batches updates within one task, but separate WebSocket messages are separate tasks. The rAF batching is across messages.

“Zustand (vs Redux Toolkit)” → notes 03

30 s: Global state avoids **prop drilling**. Zustand is a tiny store: `create()` returns a hook, and components select just the slice they need, so only those components re-render. It needs no Provider and no boilerplate. Redux Toolkit is the same idea (store, slices, actions, reducers, immutable updates) with more structure, DevTools and middleware. It’s better for big teams. **Follow-up:** *Using the store outside React?* `useStore.getState()` / `setState()`, which is how the WebSocket layer writes ticks without being a component.

“200+ strike option chain (OI, IV, Black–Scholes Greeks)” → notes 07

30 s: For one expiry, each **strike** row has a **call** and a **put**: 100 strikes below ATM + ATM + 100 above = **201**. **ITM/ATM/OTM** compares the strike to the spot. **OI** (open interest) = contracts still open. **IV** = the volatility implied by option prices, i.e. how much the market expects movement. It rises away from the money (the **smile**). Prices come from **Black–Scholes** using spot, strike, time to expiry, IV and rate, recomputed on every tick. **Greeks (one line each):** - **Delta**: price change per $1 move in the underlying (calls 0→1, puts −1→0, ≈ 0.5 ATM). - **Gamma**: how fast delta changes. Highest ATM near expiry. - **Theta**: value lost per day from time decay (negative for buyers). - **Vega**: price change per 1 point of IV.

Follow-ups: *Put–call parity?* C − P = S − K·e^(−rT). My tests check it. *Black–Scholes assumptions?* Lognormal prices, constant volatility, no jumps, European exercise. Real markets break these, which is why the smile exists.

“virtualization (react-window)” → notes 07

30 s: 201 rows × ~20 cells is thousands of DOM nodes, all updated on every tick. Virtualization renders only the rows **visible in the scroll window** plus a small overscan, about 20 rows. It uses a fixed **24 px row height** to work out which rows are visible from `scrollTop`. Scrolling recycles rows. DOM size stays constant whatever the list length. **Follow-ups:** *Downsides?* Ctrl+F can’t find off-screen rows, variable heights need measuring, and accessibility needs care. *When not to use it?* For short lists: the extra complexity doesn’t pay off.

“memoized rows”

30 s: `React.memo` skips re-rendering a component when its props are shallowly equal. `useMemo` caches an expensive result until its dependencies change. The strike table (IV/OI) is rebuilt only when the expiry changes, the header is `memo` with stable `useMemo`’d props, and react-window memoizes each row. **Follow-ups:** *Why not memo everything?* The comparison itself costs something, and it breaks silently if you pass new objects or inline functions as props each render. *`useCallback`?* `useMemo` for functions: it keeps a stable reference for memoized children.

“incremental order book … sorted maps … sequence-gap resync” → notes 06

30 s: Bids are buy orders (best = highest), **asks** are sell orders (best = lowest), and the gap between them is the **spread**. Re-downloading the whole book is too heavy, so I follow Binance’s algorithm. Open the **diff stream** (`@depth@100ms`) and **buffer** it. Fetch a **REST snapshot** with a `lastUpdateId`. Drop buffered diffs older than the snapshot, then apply each diff in order. A quantity of 0 removes the level. -

Sorted maps: each side is a `Map` price→qty for O(1) lookup, plus a **sorted price array** with **binary search** for O(log n) insert position, so the "top 12" is just a slice. It's Java's `TreeMap` built by hand, because JS has no sorted map. - **Gap resync:** every diff carries `U` (first update id) and `u` (last). The next diff must start at `lastUpdateId + 1`. If `U > lastUpdateId + 1`, messages were **missed**, the book is now wrong, so **throw it away and resync** from a new snapshot. Stale diffs (`u ≤ lastUpdateId`) are ignored.

Follow-ups: *Why buffer before the snapshot?* So no diffs are lost in the window between subscribing and the snapshot arriving. *Why not a balanced tree?* For a few thousand levels, array insert via splice is fast enough and simpler. Tree vs array is an O(log n) vs O(n) shift trade-off.

"Turned Figma designs into a responsive dark-mode UI" → notes 08

30 s: Mobile-first: base styles target phones, and `lg:` breakpoints add the 3-column desktop grid. On phones there's a **bottom tab bar** (Markets/Chart/Trade/Portfolio), and inactive panels are hidden with CSS, not unmounted, so live data and chart state survive tab switches. Colours are **design tokens** (CSS variables in Tailwind's `@theme`). Dark/light just swaps the token values under `[data-theme]`, and an inline script applies the saved theme before first paint (no flash). Also `h-dvh`, safe-area insets for notches, `:focus-visible` and `prefers-reduced-motion`. **Figma, honestly:** "I implemented it from a dark trading-app design. Colours and spacing are tokens that map one-to-one to Figma variables, with Dark/Light modes. Auto layout ≈ flexbox, and I read specs from the Inspect panel." **Follow-ups:** *Mobile-first vs desktop-first?* Start with the smallest screen's constraints and add complexity with `min-width` queries. *Why tokens?* One place to change a colour, theming for free, and a shared vocabulary with designers.

"tested P&L and order logic with Vitest + React Testing Library" → notes 09

30 s: Unit tests (Vitest) for the pure logic: P&L averaging, validation edge cases, Black–Scholes (parity, known values), the order book (apply/stale/gap) and the option chain. **Integration tests** for the trading store: place → validate → fill → match limit orders as prices move. **Component tests** with RTL render the real `OrderForm` in jsdom, find things **by role and label like a user**, type with `userEvent`, and assert on what's visible. Around **99% line coverage** of trading logic, and the suite runs in ~2 s. **Follow-ups:** *Testing pyramid?* Many unit tests, fewer integration tests, few E2E tests. *Why `getByRole`?* It mirrors users and assistive tech, survives restyling and catches accessibility issues. *Mocks?* Prefer real logic, and mock only boundaries like the network and the clock.

"an order flow with Playwright" → notes 09

30 s: Playwright drives **real Chromium** against the running app, on **desktop and a Pixel 7 viewport**. The Binance WebSocket and REST calls are **intercepted** (`routeWebSocket`, `route`), so the test controls prices. Journeys: buy at market and watch P&L move as the fake price changes, a limit order rests then fills, and the account survives a reload. Assertions **auto-wait**, so there are no sleeps and no flakiness. **Follow-up:** *Why mock Binance in E2E?* Determinism (prices change every second), no dependency on network access in CI, and the ability to create situations on demand.

“on GitHub Actions CI” → notes 10

30 s: A workflow runs on every push and PR. One job does lint → type-check + build → Vitest with coverage. A parallel job installs Chromium and runs Playwright, uploading traces if it fails. If both pass on `main`, a deploy job builds with the right base path and publishes to **GitHub Pages**. `npm ci` from the lock file, npm caching, least-privilege permissions and cancelling outdated runs.

---

4. Numbers to remember

Symbols in the watchlist	8 USDT pairs
Frame budget	16.6 ms (60 FPS)
Heartbeat	check 5 s · ping after 10 s · dead after 20 s
Backoff	1 s doubling, capped at 30 s, with 50–100% jitter
Order book	1000-level snapshot · diffs every 100 ms · top 12 shown
Option chain	201 strikes · ~20 rows rendered · 24 px rows
Candle history	500 candles per load, 6 intervals
Tests	71 Vitest (unit/integration/component) + 6 Playwright (3 journeys × 2 devices) ≈ 77 · ~99% coverage of trading logic

5. “What was hard / what would you improve?” (have one story ready)

- **Hardest bug story:** “Some networks blocked `stream.binance.com`. The client rotated to the fallback, but each new socket (chart, order book) started on the blocked URL again and wasted a second. I made the last working URL shared, so new sockets start there.” *(Or: the order book gap logic, i.e. why you must buffer before the snapshot.)*
- **Trade-off story:** “I hide mobile tabs with CSS instead of unmounting them. That keeps the chart, its WebSocket and the order book state alive, at the cost of keeping them in memory. On a 4-panel app that’s the right trade.”
- **Improvements:** a real options feed (Deribit) for IV/OI; a backend that holds API keys for real orders; move parsing to a Web Worker; a shared single WebSocket connection with multiplexed streams; visual regression tests; a PWA/offline shell.

6. Generic frontend questions they may add

1. **Virtual DOM / reconciliation?** React diffs the new element tree against the old one and applies minimal DOM changes. Keys guide list diffing.
2. **Controlled vs uncontrolled inputs?** Value in React state vs value in the DOM (read via ref).

3. **Debounce vs throttle vs rAF?** Wait for quiet / at most once per interval / once per frame.
4. **CORS?** The browser blocks cross-origin reads unless the server allows the origin. Binance's `data-api.binance.vision` allows it.
5. **Event loop?** Call stack → microtasks (Promises) → render steps (rAF, layout, paint) → macrotasks (timers, I/O, WebSocket messages).
6. **localStorage limits?** ~5 MB, synchronous, strings only, per origin. Never store secrets there.
7. **Accessibility basics?** Semantic HTML, labels, keyboard focus (`:focus-visible`), ARIA only when needed, colour contrast, reduced motion.
8. **Core Web Vitals?** LCP (loading), INP (responsiveness), CLS (layout stability).

7. Final checklist before the interview

- [ ] Say the pitch in under 60 seconds without reading.
- [ ] Draw the architecture picture from memory.
- [ ] Explain heartbeat, backoff+jitter and resubscription with the real numbers.
- [ ] Walk through the order book algorithm, including **why buffer first** and **what a gap is**.
- [ ] Define the four Greeks in one line each.
- [ ] Explain why rAF batching, virtualization and memo each reduce work (renders vs DOM nodes vs recomputation).
- [ ] Mention the two honesty points (simulated orders, modelled IV/OI) before they ask.
- [ ] Open the live demo link on your phone and show it.